

JAVA INTERVIEW PREPARATION

CHAPTER 55

DISTRIBUTED CACHING
AT SCALE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Distributed Caching at Scale

Senior / Lead Java Developer Reading Chapter

Distributed caching is often introduced as a simple performance technique: place frequently requested data in a fast store so the application does not repeatedly query the database. That description is correct, but it hides the architectural difficulty. Once a cache is shared by many application instances, it becomes another distributed system with its own data model, failure modes, capacity limits, and consistency behavior.

A senior engineer therefore does not begin with `@Cacheable`. The design begins with a workload: which operation is too slow or too expensive, what freshness the business requires, how the system behaves when the cache is unavailable, and whether the source of truth can survive a sudden return of all traffic.

```
Client -> Java service -> distributed cache -> source of truth
                        | hit: return fast
                        | miss: load, populate, return
```

The cache improves the common path, but the source of truth still owns correctness.

Start with the purpose of the cache

A cache can reduce database reads, hide the latency of a remote service, precompute an expensive result, or absorb repeated traffic for a hot object. Those are different problems. They imply different keys, lifetimes, and failure policies.

For example, caching a product description for ten minutes may be harmless. Caching an account balance for ten minutes may violate a user-visible correctness requirement. A permissions cache may be fast but dangerous if revoked access remains effective. “Can this be cached?” is therefore incomplete. The useful questions are:

- What system remains the authoritative source?
- How stale may the value be, and under which operations?
- Can absence be cached safely?
- What is the cost of a miss?
- What happens after an update?
- What happens when the cache is slow, empty, partitioned, or unavailable?

Cache hit rate alone does not prove value. A 95 percent hit rate can still be poor if the remaining misses concentrate on an overloaded database or if large cached objects consume excessive network and memory capacity.

Local, distributed, and two-level caches

An in-process cache such as Caffeine avoids a network hop and can be extremely fast. Each application instance, however, owns a separate copy. Invalidating all copies is difficult, memory is duplicated, and instances may observe different values.

A distributed cache such as Redis gives instances a shared cache state and centralizes capacity. It adds serialization, network latency, connection management, and another production dependency. It is faster than many databases, not as fast as memory inside the JVM.

A two-level design combines both:

```
Request -> L1: local JVM cache -> L2: distributed cache -> database
           microseconds           network hop           durable truth
```

The L1 layer is useful for extremely hot, read-mostly values, but it widens the consistency problem. An L2 eviction or update does not automatically remove stale L1 copies. Teams commonly use very short L1 TTLs, versioned keys, or invalidation events. The additional layer is justified only when measurements show that the distributed-cache hop is significant.

Cache access patterns

Cache-aside

In cache-aside, application code reads the cache. On a miss it reads the database and writes the result into the cache. Updates normally change the database and then invalidate the cache.

```
READ:  cache.get(key) -> miss -> database.read -> cache.put -> return
WRITE: database.update -> cache.delete
```

This pattern is common because the cache remains optional and the application controls fallback. The important race is between reads and writes. A reader can fetch an old database value, a writer can commit a new value and invalidate the key, and then the reader can write the old value into the cache. TTL limits the duration but does not eliminate the inconsistency. Versioned keys, write ordering, change events, or validation against a version may be needed for sensitive data.

Deleting after a successful database commit is generally safer than updating the cache first. If the transaction rolls back, an early cache change would publish state that never became authoritative. In Spring, invalidation can be registered for transaction completion rather than occurring inside an uncommitted method.

Read-through and write-through

With read-through, the cache provider loads missing values. With write-through, writes pass through the cache to the authoritative store. These patterns centralize policy, but correctness now depends on provider integration and transactional guarantees. They are not automatically stronger than cache-aside.

Write-behind acknowledges the application write before durable storage is updated. It can improve throughput, but queued changes may be lost, reordered, or replayed. This is suitable only when the business accepts those semantics and the queue is durable and observable.

Keys are part of the data model

A good key is deterministic, collision-free within its namespace, and contains every input that changes the result. Include tenant, locale, authorization scope, pagination, or feature version when those affect the value.

```
catalog:v3:tenant-42:product-918:locale-en-GB
```

The prefix provides a namespace; `v3` permits a schema or algorithm migration without trying to delete every old key. Tenant identity prevents data leakage. Avoid keys built from unstable object string representations or a method's default parameter naming.

Key cardinality must also be estimated. A search cache keyed by arbitrary query text and filters can have almost no reuse while consuming large memory. Normalize bounded inputs, enforce request limits, and cache only when observed reuse justifies it.

TTL, invalidation, and freshness

TTL is an expiration safety net, not a complete invalidation strategy. A short TTL improves freshness but increases misses and load. A long TTL improves hit rate but increases stale exposure. The value should follow a business freshness budget, not a convenient round number.

Event-driven invalidation can shorten stale time. After an authoritative update, publish an event that deletes or refreshes affected keys. Delivery must be retried, consumers must be idempotent, and missed events still need a TTL or reconciliation path. If a database transaction commits but event publication fails, an outbox pattern can atomically persist both the state change and the event intent.

For strict read-your-write behavior, the writer may return the committed value directly, update or invalidate the relevant key after commit, or temporarily bypass the cache using a version token. Global strong consistency usually removes much of the simplicity and performance benefit of a cache.

Preventing a cache stampede

When a popular key expires, hundreds of requests may miss simultaneously and issue the same database query. This cache stampede can overload the source precisely when the cache stops helping.

Useful controls include:

- Request coalescing: one request loads the value while others wait for the same result.
- TTL jitter: add bounded randomness so many keys do not expire together.
- Refresh ahead: refresh a popular value before hard expiry.
- Stale-while-revalidate: serve a bounded stale value while one worker refreshes it.
- Concurrency limits: cap miss traffic reaching the source.

A distributed lock can reduce duplicate work across instances, but it creates lease-expiration and ownership problems. If a loader pauses longer than its lease, another loader may acquire the lock while the first still runs. For writes to a protected resource, a monotonically increasing fencing token is stronger: the resource rejects operations carrying an older token.

Hot keys and hot partitions

One celebrity account or global configuration key can dominate traffic. Adding cache nodes may not help if a single key is served by one shard. Local L1 caching, replicas capable of serving reads, key replication, or application-level fan-out can distribute the load. The design must preserve acceptable consistency.

Large values can also become hot. A multi-megabyte object consumes bandwidth, blocks event loops during serialization, and may cause eviction pressure. Cache the smallest stable representation needed by the caller. Sometimes caching component records is better than caching a huge assembled response; sometimes the reverse reduces many round trips. Measure both paths.

Negative caching

Repeated lookups for a missing object can attack the database just as effectively as repeated hits for an existing object. A short-lived marker for “not found” prevents this penetration. The marker needs a distinct representation so that a cached absence is not confused with a cache miss.

Negative TTLs are normally shorter because the object may soon be created. Never negative-cache transient failures as permanent absence. Authorization-sensitive absence must include the authorization scope in the key or should not be shared.

Capacity, eviction, and memory

Capacity planning must include key bytes, value bytes, object metadata, allocator fragmentation, replication, and headroom—not just the serialized payload. Measure representative production data because averages hide large tails.

When Redis reaches its configured memory limit, its eviction policy determines what happens. Depending on configuration, it may reject writes or evict keys according to policies such as least-recently-used, least-frequently-used, random selection, or TTL-oriented choices. An eviction is not ordinary expiration: frequent evictions indicate that the cache is undersized, the key space is unbounded, or TTL and object-size assumptions are wrong.

```
usable memory
  = node memory
  - operating-system and process headroom
  - replication/failover headroom
  - allocator and fragmentation allowance

estimated entries
  = usable cache bytes / measured bytes per entry
```

Monitor memory usage, fragmentation, key count, evictions, expired keys, command latency, network bandwidth, and client connection saturation. A healthy hit rate can coexist with dangerous memory pressure.

Serialization and schema evolution

Java native serialization is generally a poor distributed-cache format because it is tightly coupled to classes, carries security history, and is difficult to evolve across independently deployed versions. JSON is inspectable but verbose. Protobuf, Avro, or another explicit binary schema can be smaller and safer if evolution rules are followed.

During a rolling deployment, old and new application versions coexist. A new reader must tolerate old cached data, and an old reader may encounter new data. Options include backward-compatible schemas, version fields with multiple readers, or a versioned key prefix that creates a clean namespace. Deserialization failures should be treated as misses and measured; endlessly retrying a corrupt cached value creates an outage loop.

Replication, failover, and consistency

Redis replication is commonly asynchronous. A primary can acknowledge a write before a replica receives it; if failover occurs in that interval, the acknowledged cache value may disappear. This is often acceptable because the database remains authoritative, but it must not be mistaken for durable consensus.

Commands that wait for replicas can reduce the probability of loss, but they do not turn asynchronous replication into a strongly consistent data store under every failure. The larger design question is whether losing the cache entry merely causes a miss or violates a business invariant. Business truth, idempotency records, and financial state generally belong in durable storage with suitable consistency.

In a multi-region system, a single remote cache adds cross-region latency and a broad failure dependency. Region-local caches improve latency but can diverge. Prefer region-local caches for data whose bounded staleness is acceptable, and propagate invalidations or version changes asynchronously. Route strict operations to the authoritative region or data store.

Designing for cache failure

“Fall back to the database” is safe only if the database can accept the fallback load. A complete design uses short cache timeouts, bounded connections, circuit breaking where useful, and admission control on the miss path. It may serve stale values, reduce optional features, or reject excess work rather than transferring the outage to the database.

```
cache healthy -> normal hit/miss policy
cache slow    -> short timeout; do not occupy request threads indefinitely
cache down    -> bounded fallback; protect database; degrade optional data
cache empty   -> gradual warm-up; prevent synchronized mass loading
```

Cache clients should not retry blindly. A retry multiplies load and consumes the caller's latency budget. Retry only operations and failures for which another attempt is meaningful, with a small limit and jitter. Many cache reads are better handled as a miss after one short timeout.

Spring cache abstraction: useful but not invisible

Spring's cache abstraction can keep application code concise:

```
@Cacheable(cacheNames = "products", key = "#tenantId + ':' + #productId",
    unless = "#result == null")
public ProductView find(String tenantId, long productId) {
    return repository.findView(tenantId, productId)
        .orElseThrow(ProductNotFoundException::new);
}

@CacheEvict(cacheNames = "products", key = "#tenantId + ':' + #productId")
public void update(String tenantId, long productId, ProductPatch patch) {
    repository.applyPatch(tenantId, productId, patch);
}
```

The annotations define interception policy, not distributed consistency. In the common proxy mode, self-invocation does not pass through the proxy, so calling a cached method from another method on the same object can bypass caching. Non-public methods and final/proxy details can also matter depending on configuration.

Default key generation may be insufficient for overloaded methods, tenant isolation, or schema evolution. Define explicit keys and test them. `@CachePut` always executes the method and updates the cache; `@CacheEvict` removes entries. Conditions and `unless` clauses affect whether access is cached. `sync=true` can coordinate concurrent loading for a key, but exact behavior and distribution depend on the cache provider; it should not be assumed to be a cross-cluster lock without verification.

Transaction ordering needs deliberate treatment. An annotation may evict before a surrounding transaction successfully commits, or may populate a value whose transaction later fails, depending on placement and cache manager. Integrate cache mutation with after-commit behavior when correctness requires it.

Observability and testing

Measure hit, miss, and error rates by cache and operation, but avoid unbounded key labels. Also measure load latency, miss concurrency, stale age, evictions, memory, timeouts, connection pool waits, serialization failures, and database traffic caused by misses. The user-facing metric is end-to-end latency and correctness, not merely cache response time.

Tests should cover more than a successful hit:

- miss followed by population;
- concurrent misses for the same key;

- update/read races;
- cache timeout and total outage;
- empty-cache restart and warm-up;
- eviction under memory pressure;
- old and new serialized versions during rolling deployment;
- invalidation event duplication, delay, and loss;
- failover immediately after a write;
- hot-key and large-value behavior.

Load tests should begin with a cold cache as well as a warm one. A design that survives steady-state hits but collapses after restart is not production-ready.

Common traps

“Redis is the source of truth because it is replicated.” Replication and persistence settings do not automatically provide the durability or consistency required for business records.

“A high hit rate means the cache is healthy.” It says nothing about stale correctness, large objects, hot keys, or the severity of misses.

“TTL solves invalidation.” TTL bounds staleness; it does not satisfy every freshness or read-your-write requirement.

“Fallback protects availability.” Unbounded fallback may destroy the database and extend the incident.

“A distributed lock guarantees one writer.” Leases expire and paused owners can continue. Critical resources may need fencing and authoritative validation.

“Spring annotations make caching transparent.” Keys, proxy boundaries, provider capabilities, transactions, and failure behavior remain architectural concerns.

A strong interview answer

When asked to design caching for a high-volume Java service, first define the expensive read, source of truth, freshness budget, and behavior during cache failure. Choose local, distributed, or two-level caching from measured latency and consistency needs. Define tenant-safe versioned keys, bounded values, TTL and invalidation policy, then protect misses with coalescing, jitter, stale serving, and concurrency limits. Treat the cache as disposable: the database must remain correct, but it must also be protected from full fallback load. Plan serialization evolution, rolling deployments, eviction capacity, replication semantics, and hot keys. Finally, validate cold-start, failover, invalidation races, and outage behavior while observing end-to-end latency, stale age, evictions, and database load—not hit rate alone.

Chapter summary

Distributed caching is a controlled trade: lower latency and source load in exchange for additional state, staleness, and failure modes. The best designs state those tradeoffs explicitly. They preserve business correctness in authoritative storage, keep the miss path bounded, and remain predictable when cache assumptions fail.

Revision Notes

- A cache accelerates access; it should not silently become the business source of truth.
- Define freshness, miss cost, update behavior, and outage behavior before choosing technology.
- Local caches are fastest but duplicated; distributed caches are shared but add network and serialization costs.
- A two-level cache widens invalidation and consistency concerns.
- Cache-aside reads on miss and normally invalidates after an authoritative commit.
- TTL bounds stale duration but does not replace a correctness-aware invalidation strategy.
- Versioned, tenant-aware keys support isolation and rolling schema changes.
- Prevent stampedes with coalescing, TTL jitter, refresh-ahead, stale serving, and bounded loaders.
- Negative caching protects the source from repeated misses; use a short TTL and distinct marker.
- Hot keys and large values can overload one shard despite a good overall hit rate.
- Capacity includes metadata, fragmentation, replication, and operational headroom.
- Evictions are a capacity signal, not ordinary successful expiration.
- Prefer explicit, evolvable serialization formats over Java native serialization.
- Asynchronous replication can lose acknowledged cache writes during failover.
- A cache outage needs bounded fallback so the database is not sacrificed.
- Spring cache annotations still require explicit keys, proxy awareness, provider validation, and transaction ordering.
- Measure stale age, load concurrency, errors, evictions, fallback traffic, and user latency.
- Test cold starts, races, failover, event loss, schema coexistence, and cache unavailability.